Singly Linked List – Search Element In Sorted List Recursive and Time Complexity.

Program:

```
bool searchElementInSortedListRecursive(Node **head, int key)
{
    if (*head == nullptr)
    {
        return false;
    }
    else
    {
        if ((*head)->data == key)
        {
            return true;
        }
        else
        {
            return searchElementInSortedListRecursive(&((*head)->next), key);
        }
    }
}
…..
case 11:
        {
            int key;
            cout << "Enter the element to search: ";
            cin >> key;

            if (searchElementInSortedListRecursive(&head, key))
            {
                cout << "Element found." << endl;
            }
```

```
        else
        {
            cout << "Element not found or List is Empty."<< endl;

        }

    }
```

First checking is whether head pointer value is null i.e. List is empty :

Pictorially depicted as:

Node * head

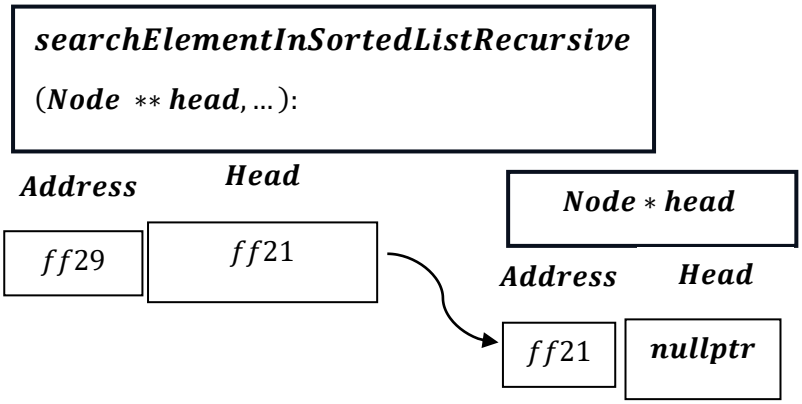
Address	***Head***
$ff21$	***nullptr***

```
        if (searchElementInSortedListRecursive(&head, key))
        {
            cout << "Element found." << endl;
        }

        else
        {
            cout << "Element not found or List is Empty."<< endl;

        }
```

$\boldsymbol{if(searchElementInSortedListRecursive(ff21, 3))\{\ldots\}else\{\ldots\};}$

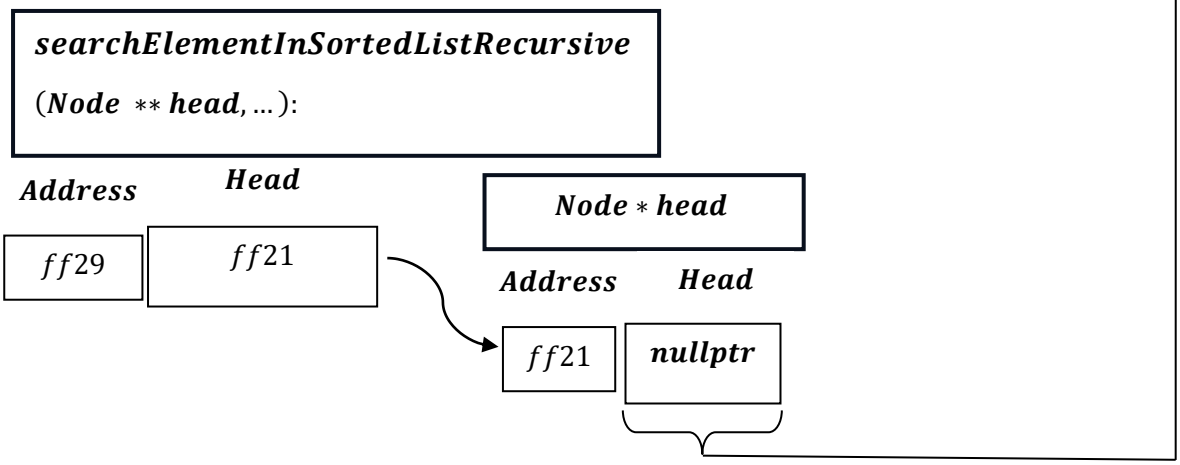
pointer head's address passed is $ff21$ in function call (pass by reference) and key is : **3** .

```
bool searchElementInSortedListRecursive(Node **head, int key){…}
```



$if(*\ head == nullptr)\ if\ it\ is\ true$

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the searchElementInSortedListRecursive function.



```
    if (*head == nullptr)
    {
        return false;
    }
```

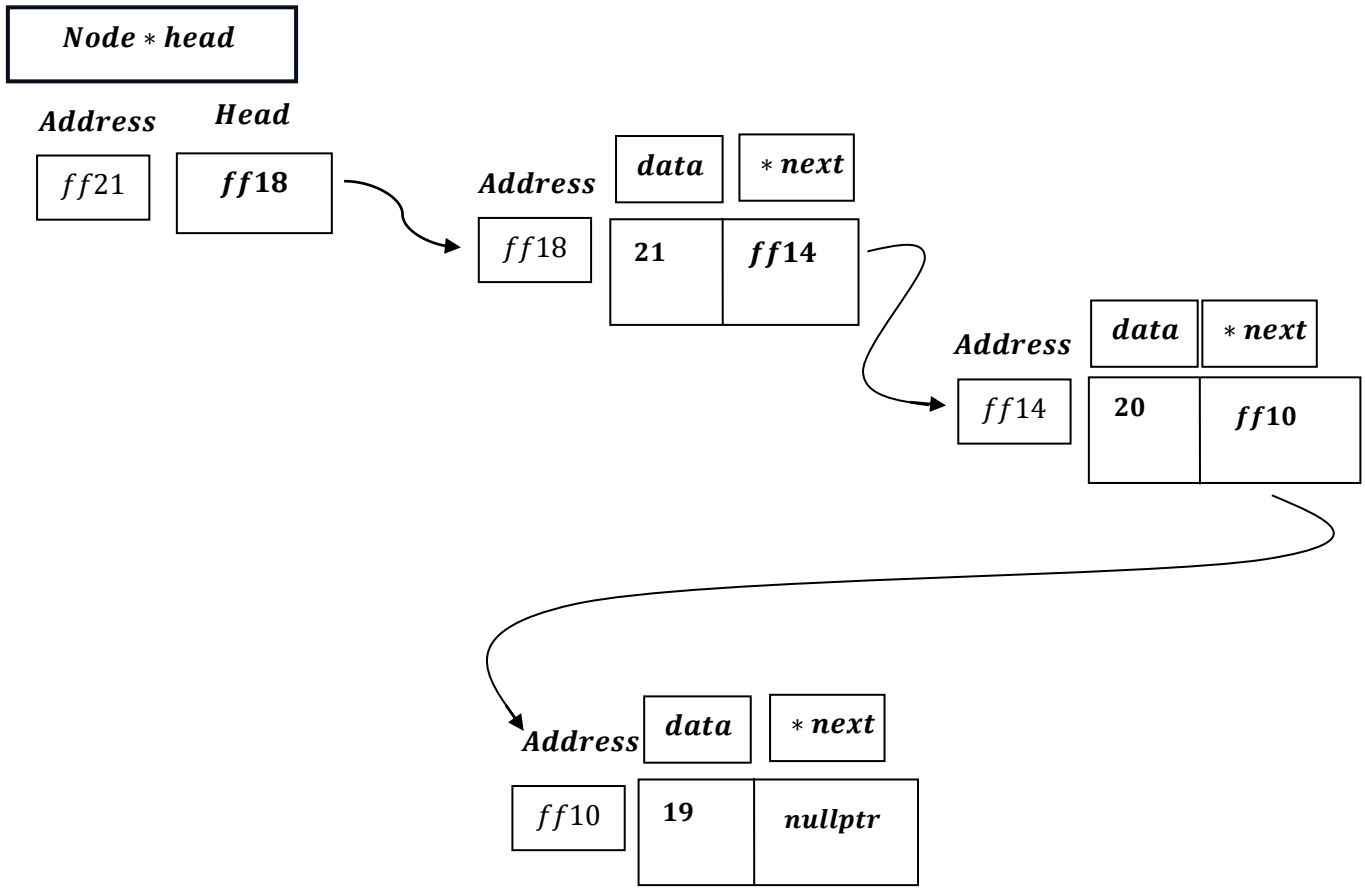
If List is Empty , then , it returns false. And as it returns false :

```
            if (searchElementInSortedListRecursive(&head, key))
            {
                cout << "Element found." << endl;
            }

            else
            {
                cout << "Element not found or List is Empty."<< endl;

            }
```

It prints "Element Not Found or List is Empty."

Now if Linked List is not Empty:

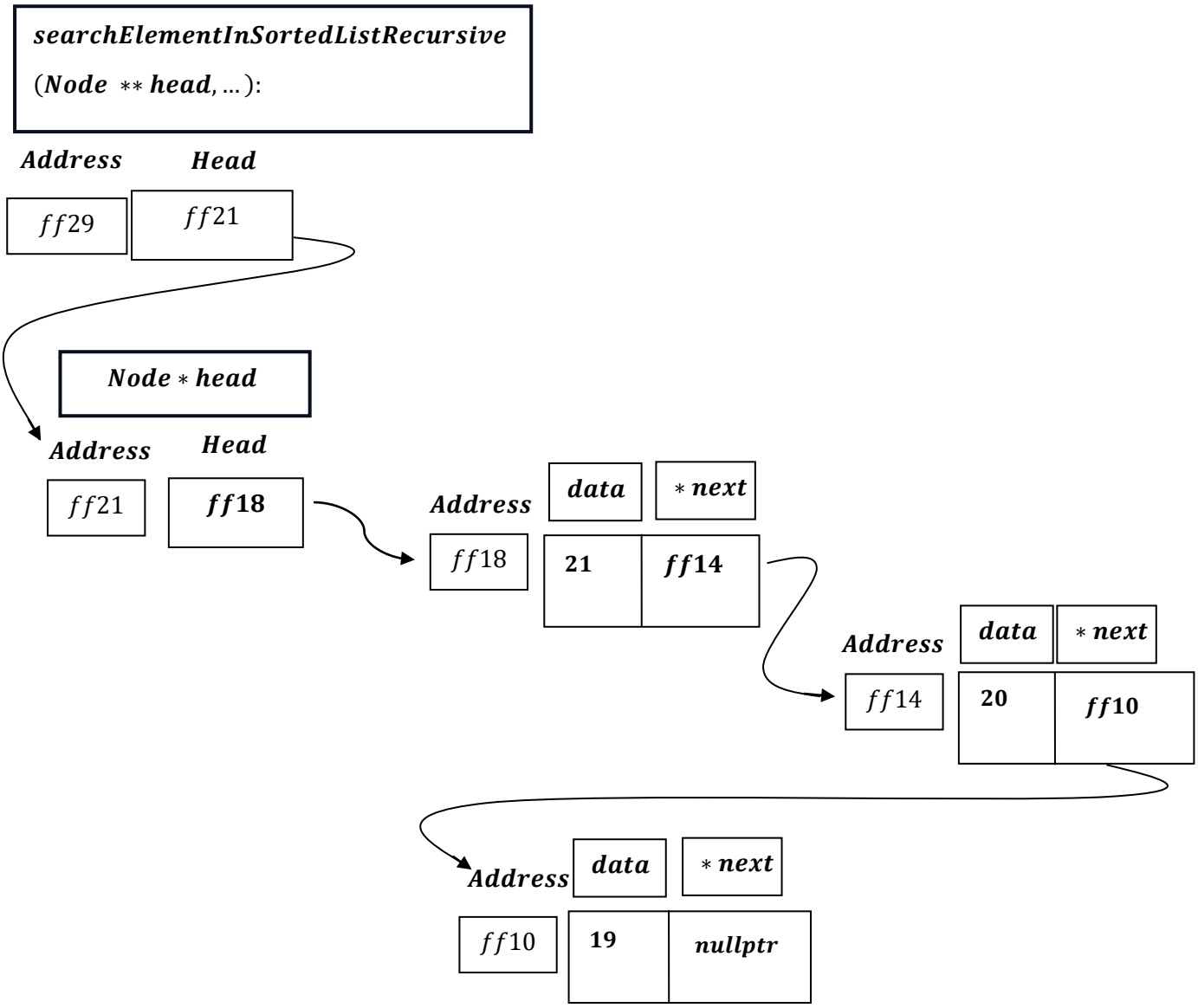
We have linked list :



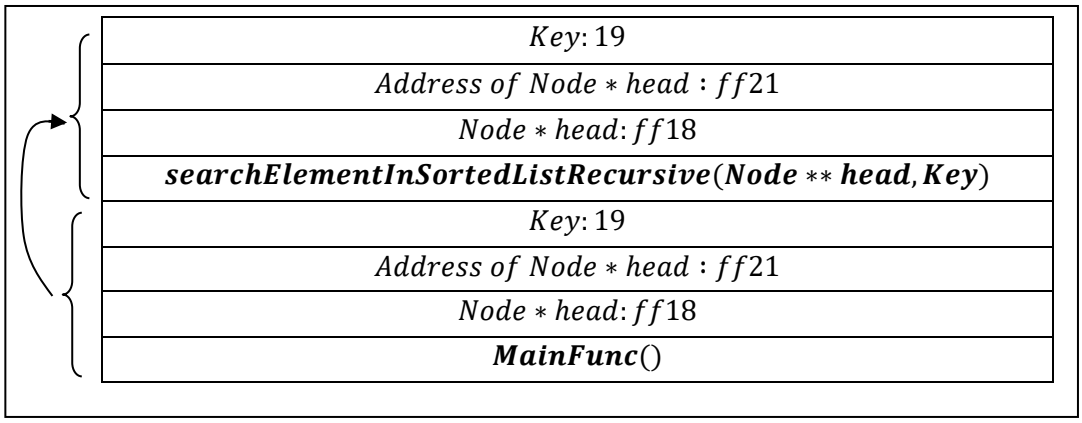

```
            if (searchElementInSortedListRecursive(&head, key))
            {
                cout << "Element found." << endl;
            }
            else
            {
                cout << "Element not found." << endl;
            }
```

$if(searchElementInSortedListRecursive(ff21, 19))\{\ldots\}else\{\ldots\}; i.e., pointer\ head's$ $address\ passed\ is\ ff21\ in\ function\ call\ (pass\ by\ reference) and\ key\ is: 19$.

```
bool searchElementInSortedListRecursive (Node **head, int key){…}
```



Stack Push 1



```
bool searchElementInSortedListRecursive(Node **head, int key)
{
    if (*head == nullptr){….}
    else
    {
        if ((*head)->data == key)
        {
            return true;
        }
        else
        {
            return searchElementInSortedListRecursive(&((*head)->next), key);
        }
    }
}
```

$Here\ like\ iterative\ , we\ have\ not\ taken\ `current`\ pointer\ , but\ we\ have\ used\ directly$

$Node * head\ , and\ we\ will\ view\ how\ it\ have\ been\ used:$

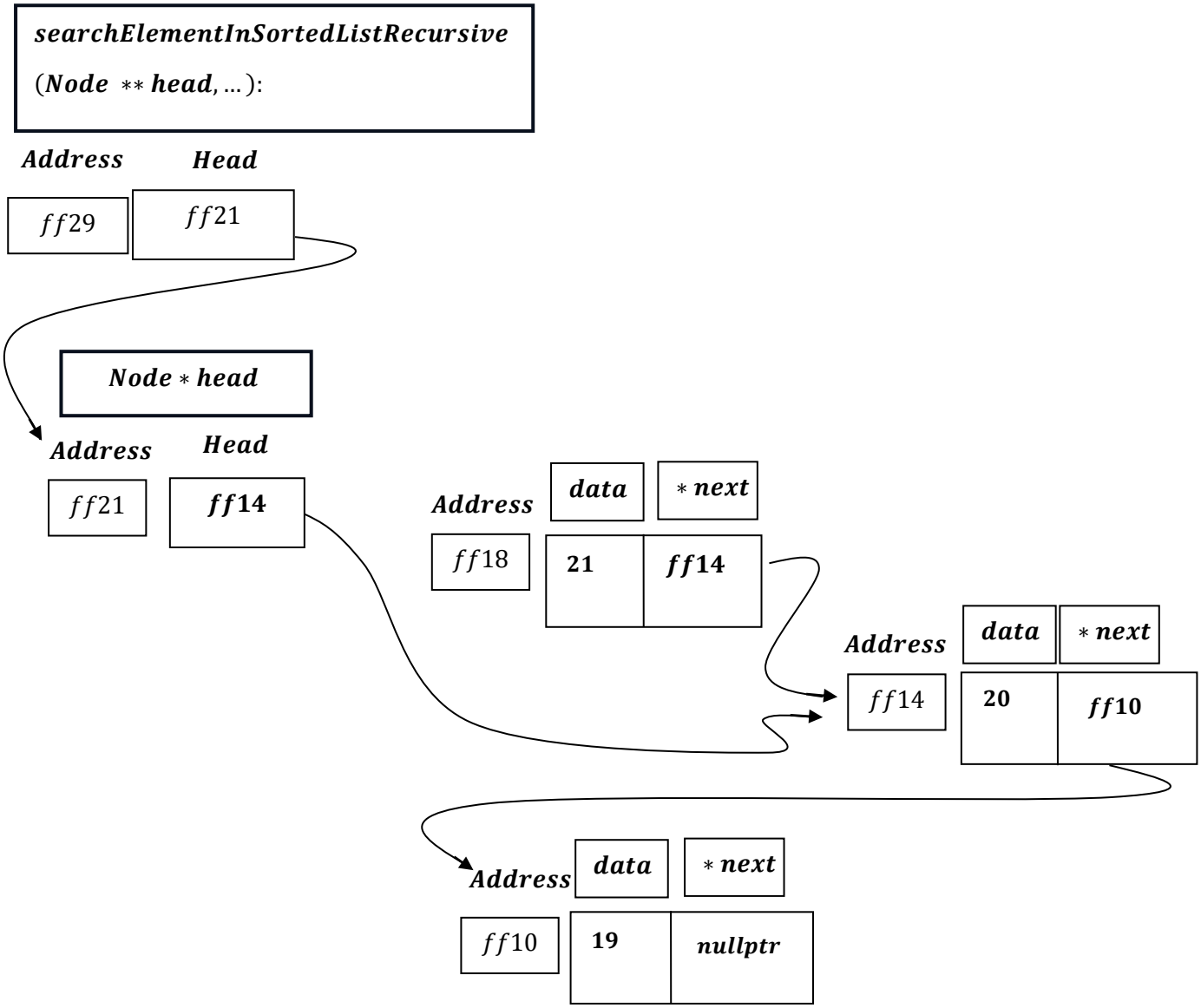
$If\big((*head) \rightarrow data == key\big)\ is\ false\ as\ (*head) \rightarrow data\ is\ 21\ and\ Key\ is\ 19.$

$Hence\ , we\ have\ recursive\ call:$

$searchElementInSortedListRecursive\big(\&\big((*head) \rightarrow next\big), key\big);$

$i.e.\ (*head) \rightarrow next\ is\ ff14\ and\ \ \&\big((*head) \rightarrow next\big)\ is\ ff21\ and\ key\ is\ 19$

$, hence\ \ now\ it\ will\ be:$



Stack Push 2

$Key: 19$
$Address\ of\ Node * head\ [\&((*head) \rightarrow next)]: ff21$
$(*head) \rightarrow next\ \ i.e. Node * head: ff14$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

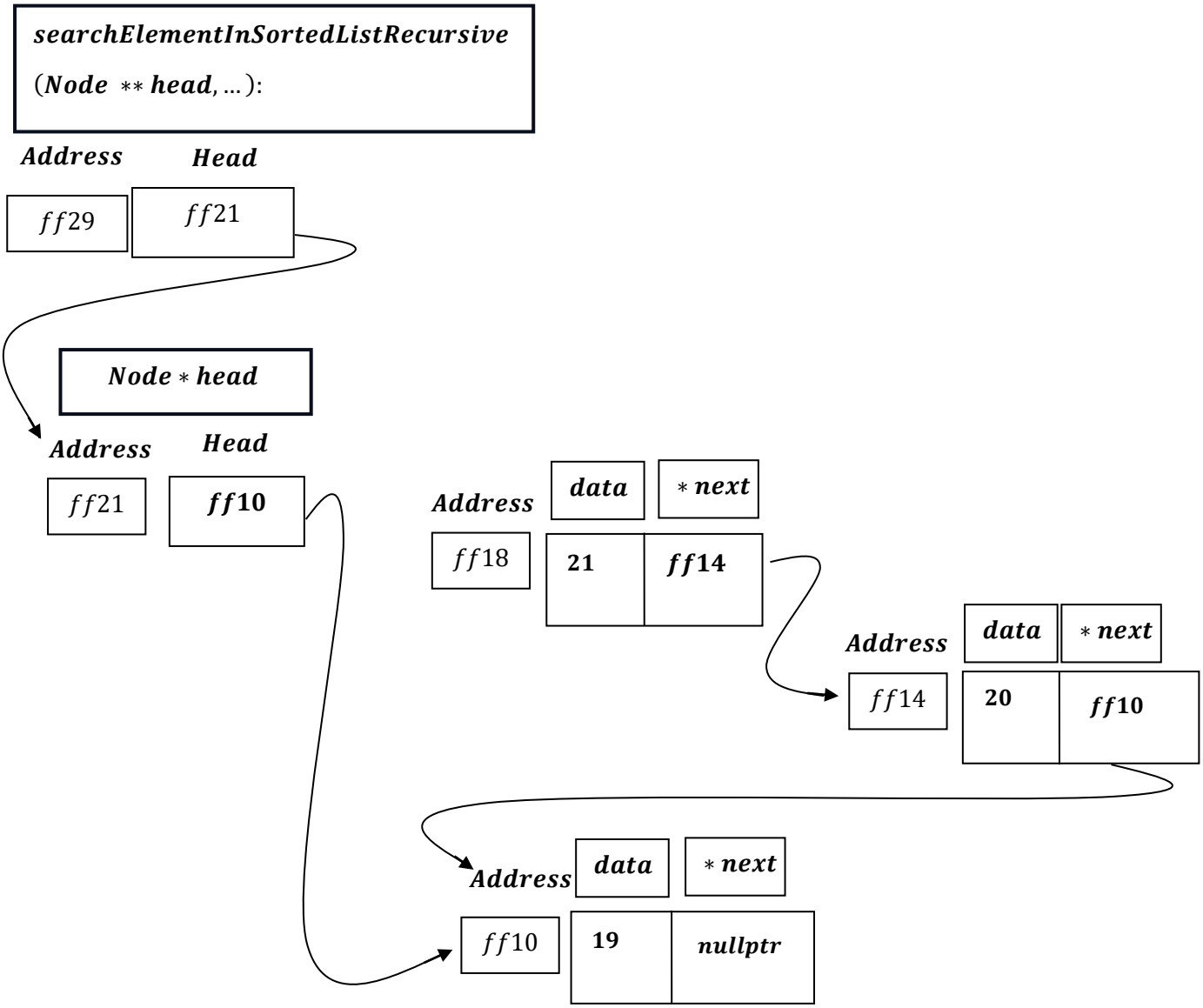
$\boldsymbol{If\big((*\,head) \rightarrow data == key\big)\ is\ false\ as\ (*\,head) \rightarrow data\ is\ 20\ and\ Key\ is\ 19}.$

$\boldsymbol{Hence\ ,again\ we\ have\ recursive\ call}$:

$\boldsymbol{searchElementInSortedListRecursive}\big(\&\big((*\,head) \rightarrow next\big), key\big)$;

$\boldsymbol{i.e.}\ (*\,head) \rightarrow next\ is\ ff10\ and\ \ \&\big((*\,head) \rightarrow next\big)\ is\ ff21\ and\ key\ is\ 19$

$\boldsymbol{,hence\ \ now\ it\ will\ be}$:



Stack Push 3

$Key: 19$
$Address\ of\ Node * head\ [\&((*head) \rightarrow next)]: ff21$
$(*head) \rightarrow next\ \ i.e. Node * head: ff10$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head\ [\&((*head) \rightarrow next)]: ff21$
$(*head) \rightarrow next\ \ i.e. Node * head: ff14$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

$\boldsymbol{If((*head) \rightarrow data == key)\ is\ true\ as\ (*head) \rightarrow data\ is\ 19\ and\ Key\ is\ 19}$.

Here search ends but head already pointing to last node or tail node , not the head node , hence now again, we pop the values that will make the Node * head , will point to head node of the linked list.

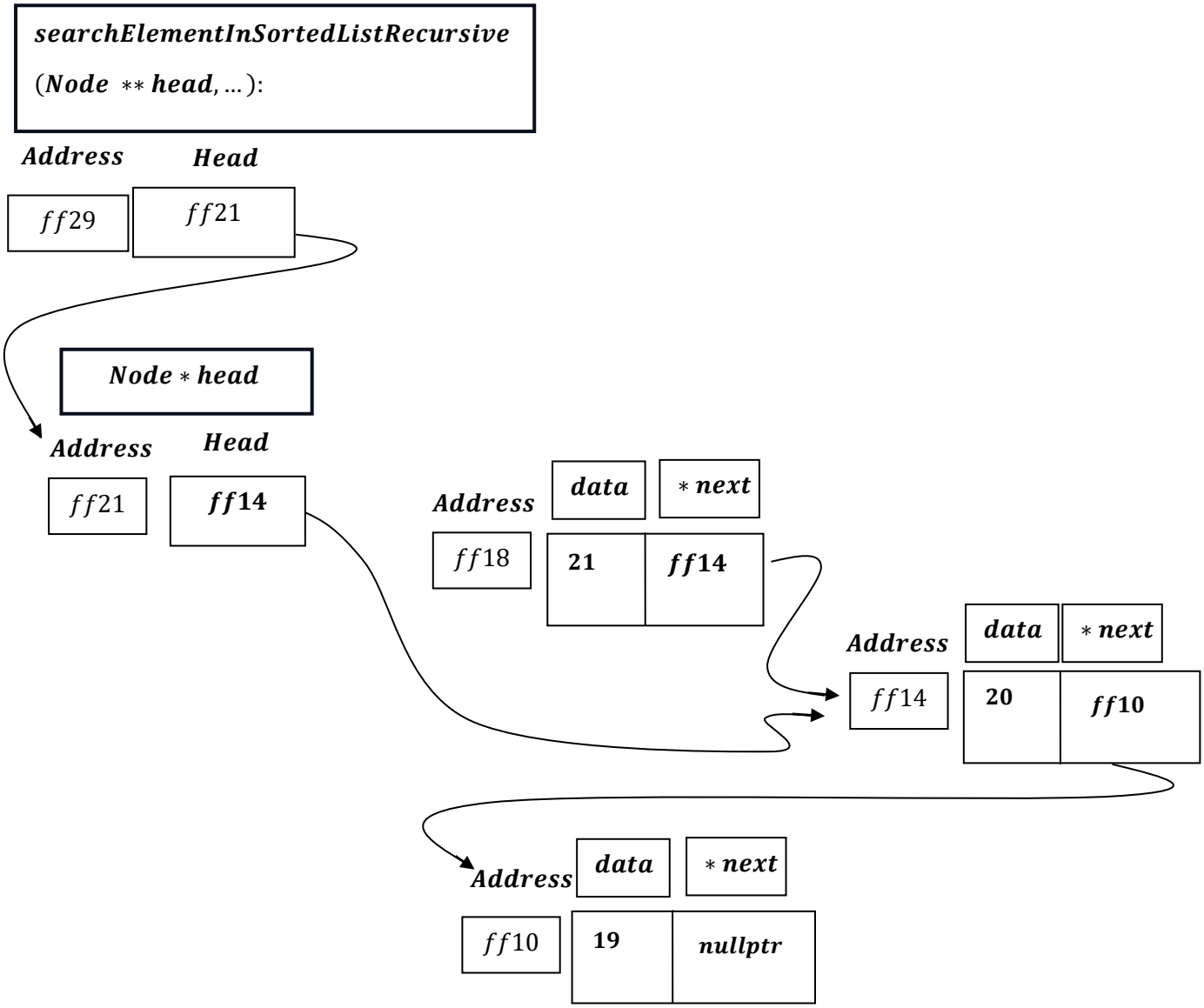
Stack Pop 1

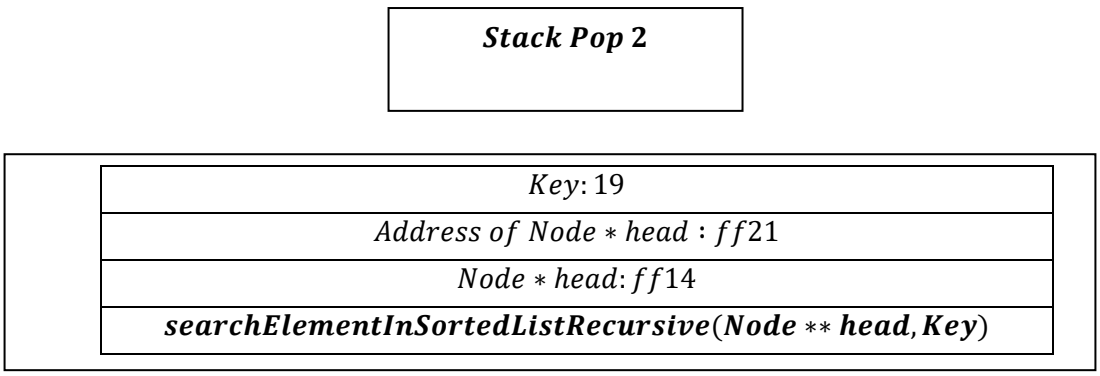
$Key: 19$
$Address\ of\ Node * head: ff21$
$Node * head: ff10$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$

This portion of the stack frame gets popped out and deallocated and remaining stack frame we have :

Key: 19
$Address\ of\ Node * head$: $ff21$
$Node * head$: $ff14$
$\boldsymbol{searchElementInSortedListRecursive}(\boldsymbol{Node} ** \boldsymbol{head}, \boldsymbol{Key})$
Key: 19
$Address\ of\ Node * head$: $ff21$
$Node * head$: $ff18$
$\boldsymbol{searchElementInSortedListRecursive}(\boldsymbol{Node} ** \boldsymbol{head}, \boldsymbol{Key})$
Key: 19
$Address\ of\ Node * head$: $ff21$
$Node * head$: $ff18$
$\boldsymbol{MainFunc}()$

***Hence as we have*, *Address of Node* $*$ *head* : $ff21$ *and Node* $*$ *head*: $ff14$, *hence Node* $*$ *head point to memory location with the address* $ff14$, *as shown below*:**

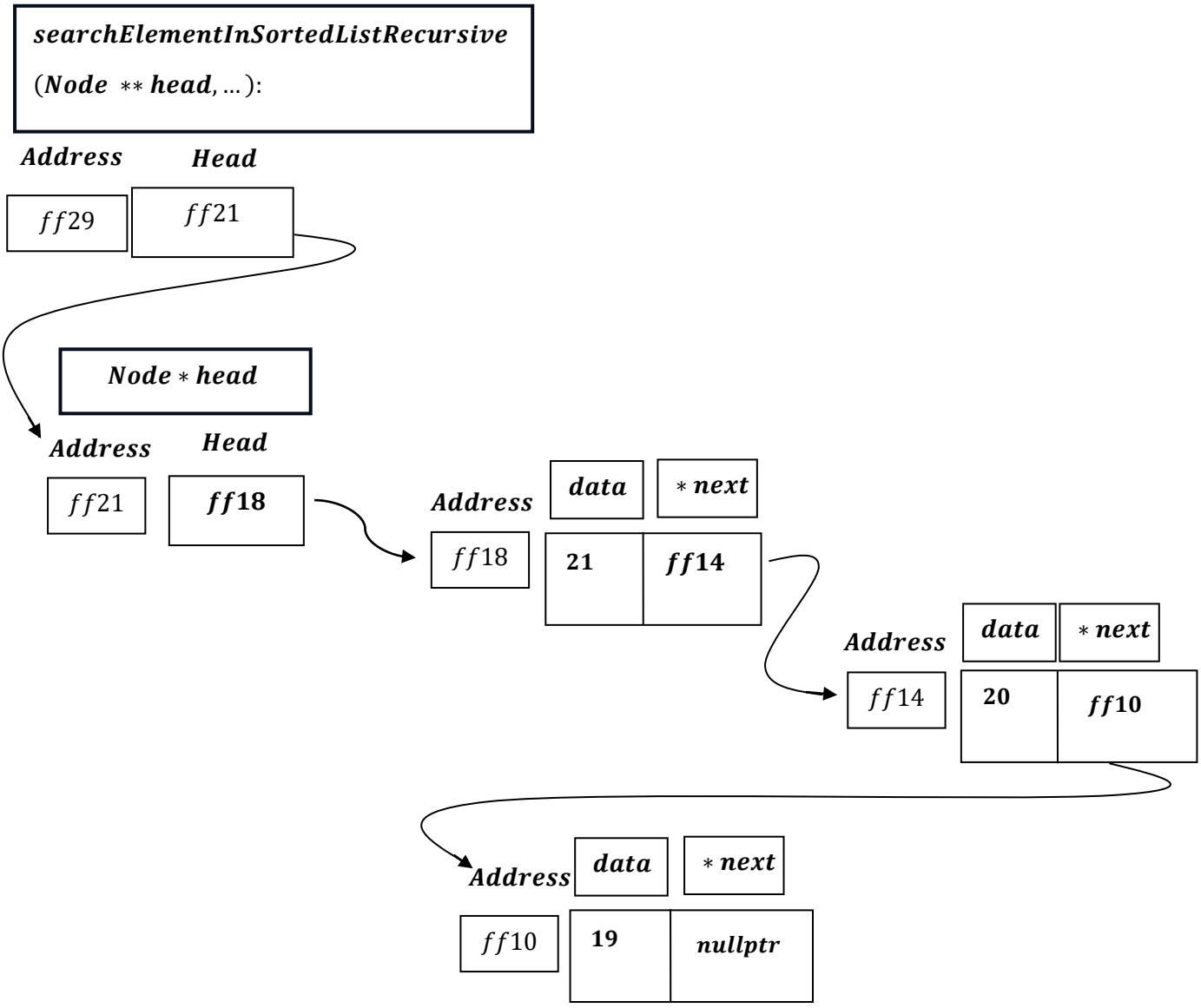

searchElementInSortedListRecursive
(Node ** head, ...):
Address
Head
ff29
ff21
Node * head
Address
Head
ff21
ff14
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10
Address
data
* next
ff10
19
nullptr


Stack Pop 2
Key: 19
Address of Node * head : ff21
Node * head: ff14
searchElementInSortedListRecursive(Node ** head, Key)

This portion of the stack frame gets popped out and deallocated and remaining stack frame we have :

Key: 19
$Address\ of\ Node * head$: $ff21$
$Node * head$: $ff18$
$\boldsymbol{searchElementInSortedListRecursive}(\boldsymbol{Node} ** \boldsymbol{head}, \boldsymbol{Key})$
Key: 19
$Address\ of\ Node * head$: $ff21$
$Node * head$: $ff18$
$\boldsymbol{MainFunc}()$

Hence as we have, Address of Node * head : ff21 and Node * head: ff18 , hence Node * head point to memory location with the address ff18 , as shown below:



Stack Pop 3

$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$

This portion of the stack frame gets popped out and deallocated and main function stack frame we have :

$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$

That is after the function module ends , all temporary pointer variables created gets destroyed, i.e.

searchElementInSortedListRecursive (Node ** head, ...):

Address	***Head***
$ff29$	$ff21$

Gets destroyed after, execution of the function finished and when function exits.

<u>Now , when there is case `Key Not Found`:</u>

Say key is 18 , then we go back to push:

$\boldsymbol{If((*head) \rightarrow data == key)}$ ***is false as*** $\boldsymbol{(*head) \rightarrow data}$ ***is 19 and Key is 18.***

Hence , again we have recursive call :

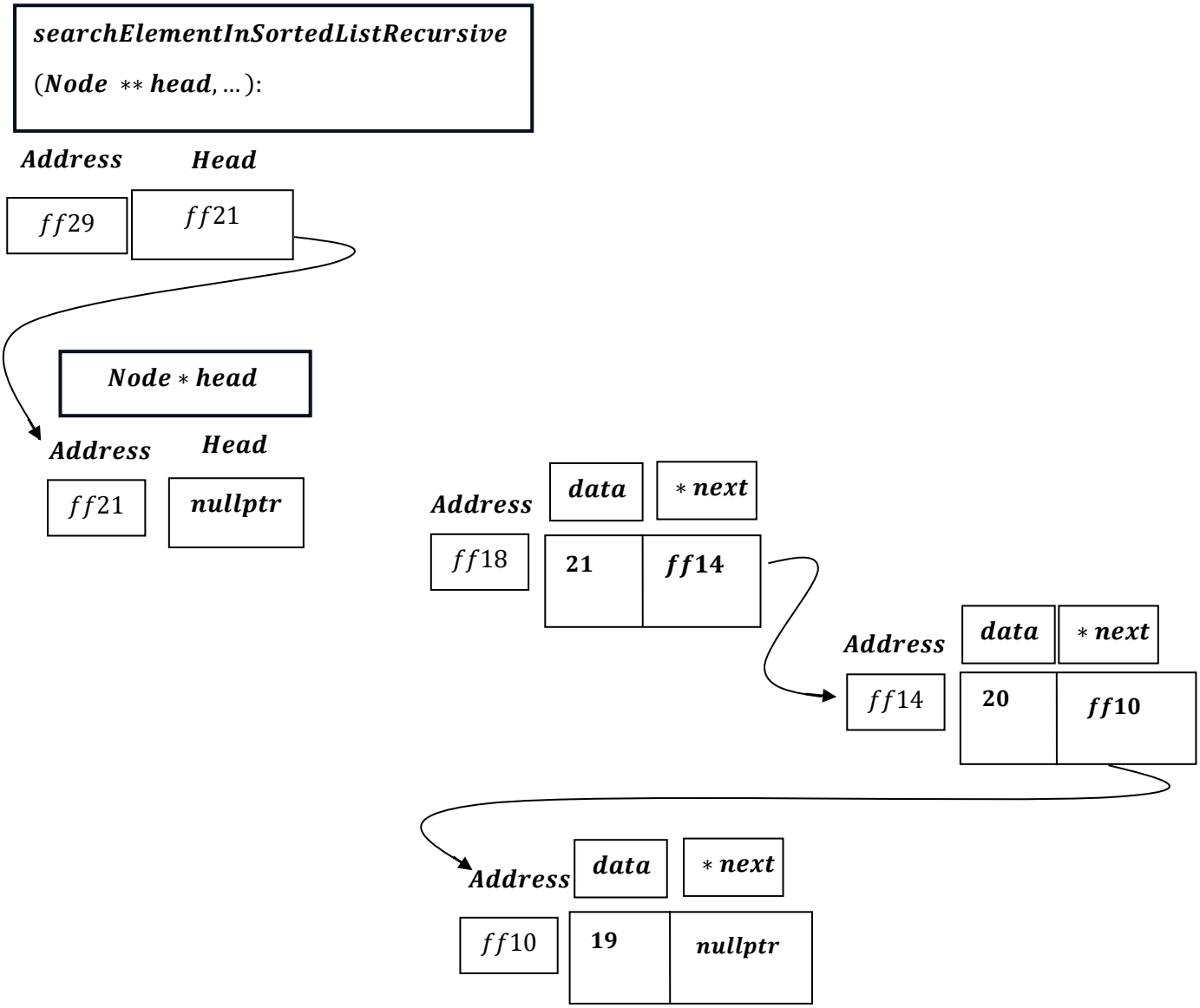
$\boldsymbol{searchElementInSortedListRecursive(\&((*head) \rightarrow next), key)}$;

$\boldsymbol{i.e.}\ (*\ \boldsymbol{head}) \rightarrow \boldsymbol{next\ is\ ff10\ and}\ \ \&\big((*\ \boldsymbol{head}) \rightarrow \boldsymbol{next}\big)\ \boldsymbol{is}$ `**nullptr**` $\boldsymbol{and\ key\ is}\ \mathbf{19}$

$,\boldsymbol{hence\ \ now\ it\ will\ be}$:

Stack Push 4

$Key: 19$
$Address\ of\ Node * head\ \big[\&\big((* head) \rightarrow next\big)\big]: ff21$
$(* head) \rightarrow next\ \ i.e. Node * head: nullptr$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head\ \big[\&\big((* head) \rightarrow next\big)\big]: ff21$
$(* head) \rightarrow next\ \ i.e. Node * head: ff10$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head\ \big[\&\big((* head) \rightarrow next\big)\big]: ff21$
$(* head) \rightarrow next\ \ i.e. Node * head: ff14$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$



Next,

```
    if (*head == nullptr)
    {
        return false;
    }
```

As head contains nullptr , it returns false. And as it returns false :

```
if (searchElementInSortedListRecursive(&head, key))
{
    cout << "Element found." << endl;
}

else
{
    cout << "Element not found or List is Empty."<< endl;
}
```

It prints "Element Not Found or List is Empty."

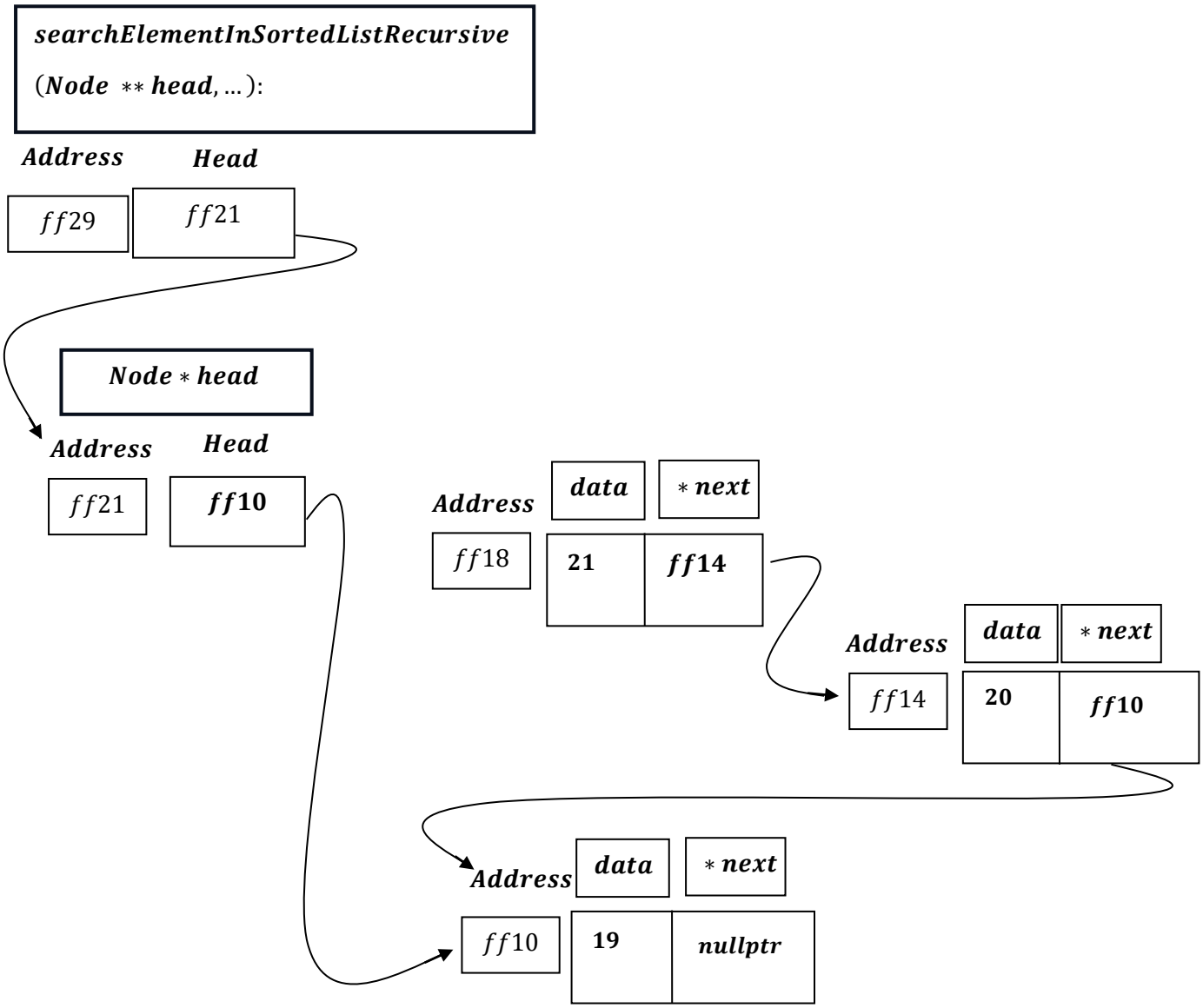
Now, question is whether Node * head = nullptr remain forever, no its absolutely not, as stack pop follows:

Stack Pop 1

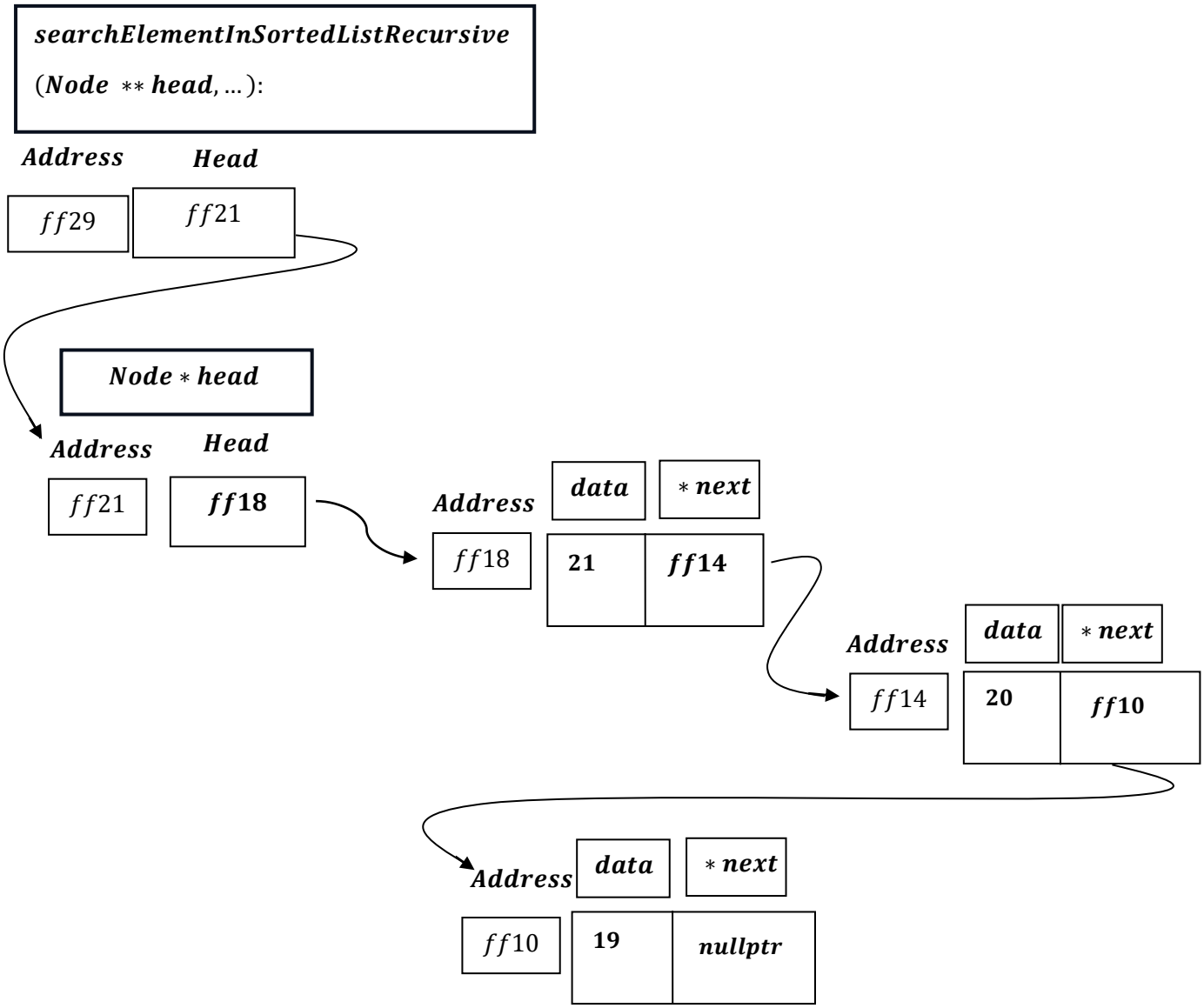
$Key: 19$
$Address\ of\ Node * head\ [\&((*head) \rightarrow next)]: ff21$
$(*head) \rightarrow next\ \ i.e. Node * head: nullptr$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$

This portion of the stack frame gets popped out and deallocated and remaining stack frame we have :

$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff10$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head: ff21$
$Node * head: ff14$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{searchElementInSortedListRecursive(Node ** head, Key)}$
$Key: 19$
$Address\ of\ Node * head : ff21$
$Node * head: ff18$
$\boldsymbol{MainFunc()}$



And , as pop finishes , head pointer again start pointing to Head Node i.e.



The searchElementInSortedListRecursive function step by step explanation:

1. *Initial Call*:

The function is initially called with the head of the list and the `key` to search for.

2. *First Node Check*:

– The function checks if the list is empty (`*head == nullptr`). If it is, it returns `false`.

– If the list is not empty, it checks if the data in the current node ((*head) → data) matches the `key`.

– If it matches, it returns `true`.

– If it does not match, it makes a recursive call with the next node.

3. *Recursive Calls*:

– The function continues to make recursive calls, each time moving to the next node in the list.

– This process repeats until either:

– The `key` is found in a node, in which case `true` is returned.

– The end of the list is reached (`*head == nullptr`), in which case `false` is returned.

Time Complexity of searchElementInSortedListRecursive Function:

```
bool searchElementInSortedListRecursive(Node **head, int key)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return false;
    }
    else
    {
        if ((*head)->data == key)
        {
            return true;
        }
        else
        {
            return searchElementInSortedListRecursive(&((*head)->next), key);
        }
    }
}
```

$O(1)$

Takes constant time `1` at each time, function runs.

1. *Best Case*:

1) *When List is empty*:

– *The function first checks if the linked list is empty by examining the `*head` pointer.*

– *If `*head` is `nullptr`, it means the list is empty.*

$-$ *An error message "Error: List is empty." is printed to the console.*

$-$ *The function returns `false` to indicate that the key was not found because the list is empty.*

This whole process takes constant amount of time i.e. $\Omega(1)$.

2)*When Key present at Head node*:

$-$*Now head pointer doesnot carries `Null Pointer` value, hence it checks at if condition first , such checking takes constant amount of time i.e.* $O(1)$.

$-$*Next it moves to else part and head pointer points to memory location, checks value inside data variable present in memory location equals to the key, and as it is the best case , it gets matched and return `true`.*

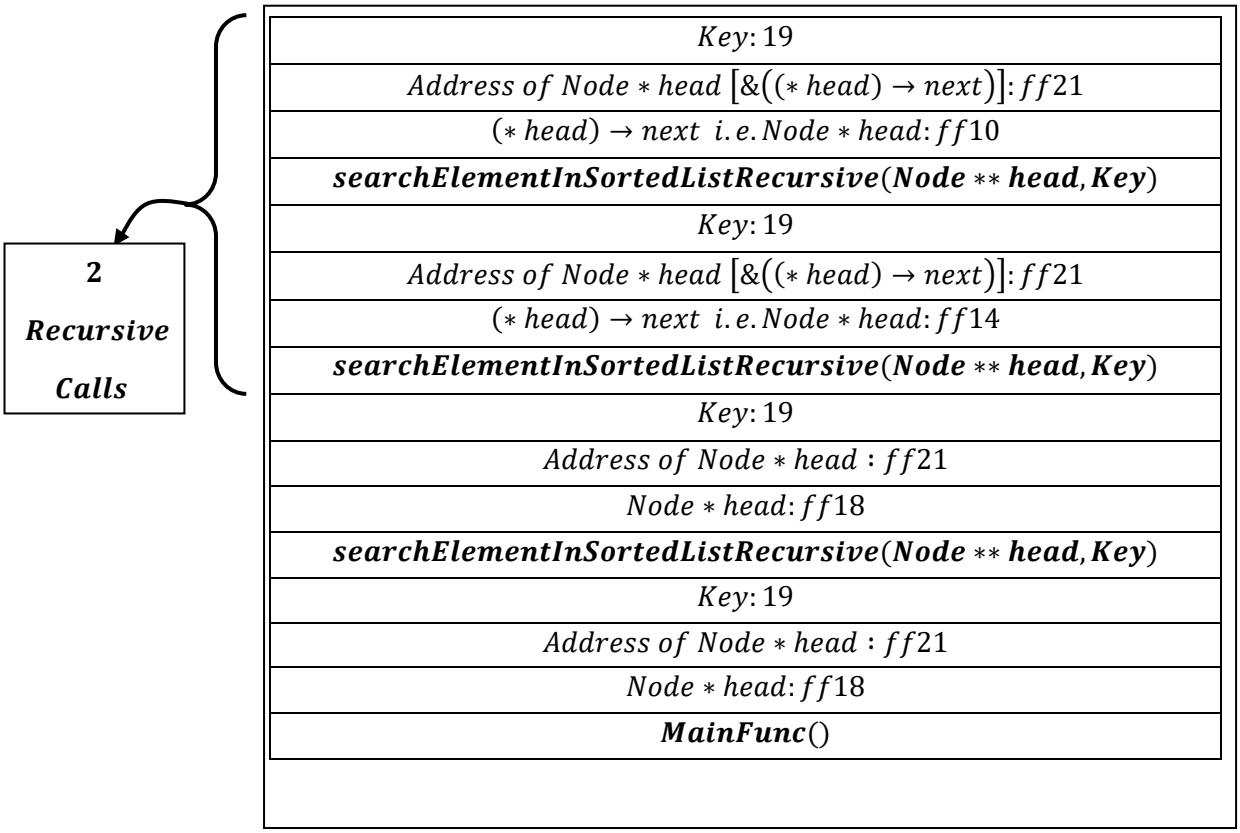
This whole process takes constant amount of time i.e. $\Omega(1)$.

2. *Worst Case*:

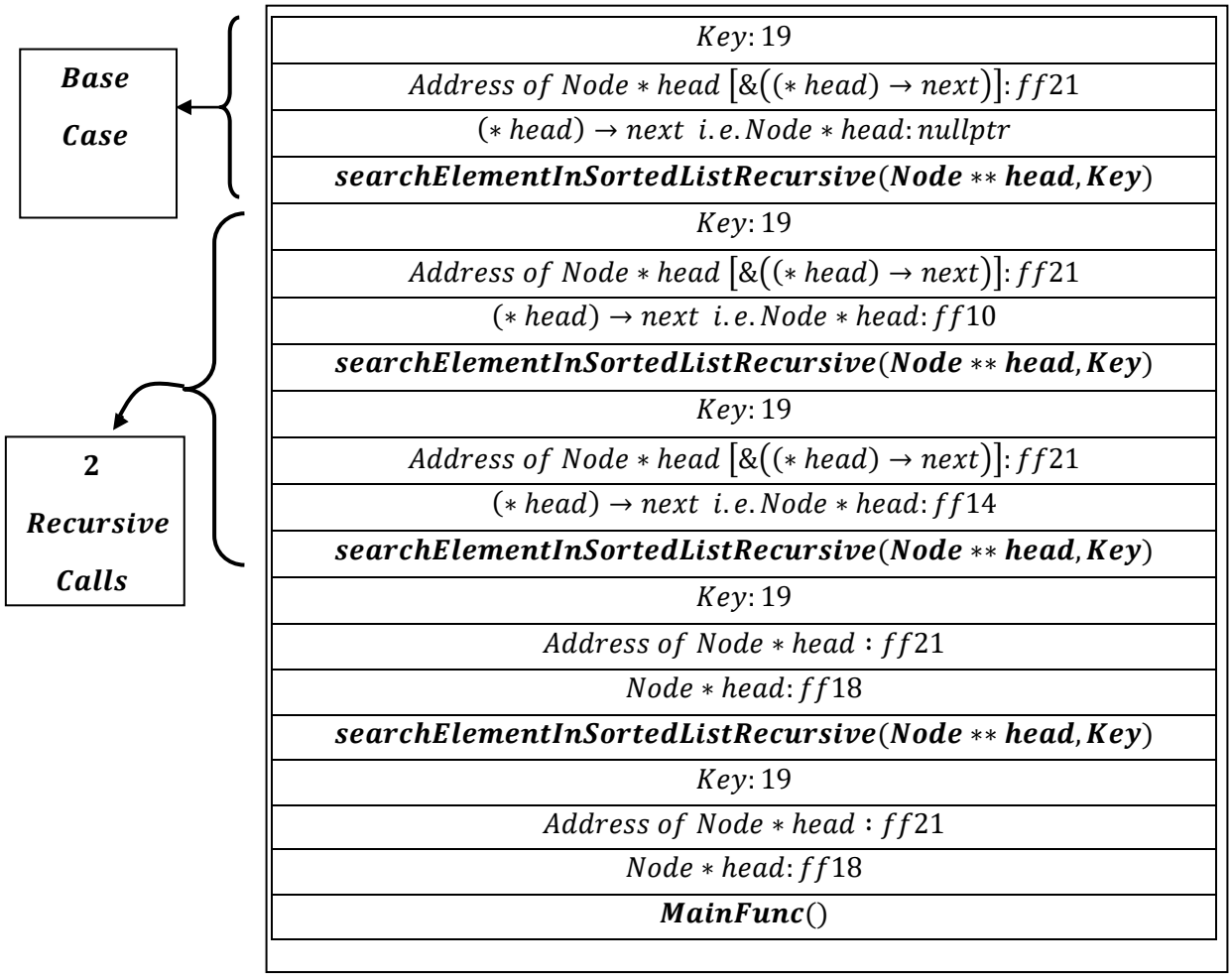
While calculating worst case , we should note that, list is not empty.
Condition: `Key Not Found` or `found at last Node` of the list.

If we recall earlier stack push, when length of the linked is 3 :

1)*When `Key` found at last Node of the list, we have* :



2)*When `Key` is absent*:



$Though\ , three\ recursive\ calls\ , the\ last\ call\ assigns\ Node * head = nullptr\ ;$
$Hence\ , its\ our\ base\ case\ .$

$Therefore\ Key\ is\ searched\ first\ takes\ constant\ time\ i.e.$
$if\big((* head) \rightarrow data == Key\big)\{\ldots\} takes\ O(1)\ time\ complexity.$

$And\ then, Key\ is\ searched\ in\ N-1\ nodes\ takes\ T(N-1)\ recursive\ calls.$

$And\ each\ time\ of\ this\ recursive\ calls\ , the\ if\ condition\ checks\ i.e.$
$if\big((* head) \rightarrow data == Key\big)\{\ldots\} takes\ O(1)\ , time\ complexity.$

$Now, if\ `Key\ is\ not\ found`\ another\ call\ executes\ which\ runs\ our\ base\ case\ , which$
$ends\ the\ recursive\ call\ takes : O(1), time\ complexity.$

$Hence\ it\ lead\ us\ two\ approches\ like\ we\ did\ in\ factorial:$

$We\ can\ say:$

$T(N) = T(N-1) + 1\ , this\ one\ additional\ `1`\ is\ for\ checking\ whether\ data\ found\ or$
$not\ i.e.\ for\ if\big((* head) \rightarrow data == Key\big)\{\ldots\}\ .$

$Also, there\ can\ be\ another\ approach\ , one\ can\ say\ when\ `Key\ is\ not\ found`\ :$

$T(N) = T(N-1) + 1, this\ one\ additional\ `1`\ is\ for\ base\ case\ , when\ ,$
$Node * head = null;$

$Therefore\ , generalised\ recurisive\ form\ we\ get:$

$$T(N) = \begin{cases} 1 & , for\ n = 0\ , as, head\ is\ nullptr. \\ 1 & , for\ key\ is\ found\ at\ n = 1\ , i.e.\ in\ first\ node. \\ T(n-1) + 1 & , for\ key\ is\ found\ at\ (n > 1). \end{cases}$$

***By Substition Method*(1):**

$T(n) = T(n-1) + 1$

$Therefore\ , T(n-1) = T(n-2) + 1.$

$Substituting\ this\ in\ T(n)\ we\ get$:

$T(n) = (T(n-2) + 1) + 1$

$Now\ , T(n-2) = T(n-3) + 1.$

$Substituting\ this\ in\ T(n)\ we\ get$:

$T(n) = \big((T(n-3) + 1)\ + 1\big)\ + 1$

$Now\ if\ it\ runs\ upto\ `i`\ times\ we\ get$:

$Now\ , T(n-i) = T(n-i) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$$T(n) = T(n-i) + 1 + 1 + 1 + \cdots i\ times$$

$When\ i = n - 1\ we\ get$:

$$T(n) = T\big(n - (n-1)\big) + 1 + 1 + 1 + \cdots (n-1) times$$

$$= T(n - n + 1) + 1 + 1 + 1 \ldots . (n-1) times$$

$$= T(1) + 1 + 1 + 1 \ldots .\ \ (n-1) times$$

$And\ we\ know, T(1) is\ 1\ and\ as\ 1\ is\ added\ to\ 1 + 1 + 1 + \cdots (n-1) times\ \ it\ becomes$:

$$= 1 + 1 + 1 + \cdots n\ times$$

$i.e. 1 \times n = n\ and\ time\ complexity\ is$:

$= O(n)$.

<u>$By\ Substition\ Method(2)$:</u>

$$T(n) = T(n-1) + 1$$

$Therefore\ , T(n-1) = T(n-1-1) + 1 = T(n-2) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$$T(n) = (T(n-2) + 1) + 1$$

$Now\ , T(n-2) = T(n-2-1) + 1 = T(n-3) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$$T(n) = \big((T(n-3)+1) + 1\big) + 1$$

$Now\ if\ it\ runs\ upto\ `i`\ times\ we\ get$:

$Now\ , T(n-i) = T(n-i-1) + 1$

$Substituting\ this\ in\ T(n)\ we\ get$:

$$T(n) = T(n-i-1) + 1 + 1 + 1 + \cdots i\ times$$

$When\ i = n-1\ we\ get$:

$$T(n) = T(n-(n-1)-1) + 1 + 1 + 1 + \cdots (n-1) times$$

$$= T(n-n+1-1) + 1 + 1 + 1 \ldots . (n-1) times$$

$$= T(0) + 1 + 1 + 1 \ldots .\ \ (n-1) times$$

$And\ we\ know\ T(0) = 1\ along\ with\ T(1) is\ 1\ and\ as\ 1\ is$
$added\ to\ 1 + 1 + 1 + \cdots (n-1) times\ \ it\ becomes$:

$$= 1 + 1 + 1 + \cdots n\ times$$

$i.e. 1 \times n = n\ and\ time\ complexity\ is$:
$= O(n)$.

<u>*By Recurrence Tree Method*</u>:

$$T(N) = \begin{cases} 1 & , for\ n = 0\ , as, head\ is\ nullptr. \\ 1 & , for\ key\ is\ found\ at\ n = 1\ , i.e. in\ first\ node. \\ T(n-1) + 1 & , for\ key\ is\ found\ at\ (n > 1). \end{cases}$$

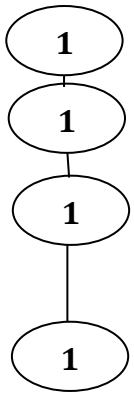
$We\ know$:

$$t_n = \begin{cases} 1 & for\ n = 1 \\ t_{n-1} + a & for\ n > 1 \end{cases}$$

$When\ a\ is\ 1\ , problem\ size\ becomes\ 1\ i.e. constant.$

$Therefore, the\ recurrence\ tree\ for\ this\ recurrence\ equation$
$would\ be\ shown\ below$:

$Level$	$No. of\ problems$	$Problem\ Size$	$Work\ done = Problem\ Size \times No. of\ Problems$
0	1	1	$1 \times 1 = 1$
1	1	1	$1 \times 1 = 1$
2	1	1	$1 \times 1 = 1$
. .	. .	. .	. .
$n - 1$	1	1	$1 \times 1 = 1$



$One\ can\ observe\ that\ the\ work\ done\ at\ every\ level\ is\ 1\ and\ the\ size\ of\ the\ problem$
$is\ reduced\ by\ a\ factor\ of\ 1\ at\ every\ level$.

$Therefore, at\ the\ level\ n - 1, the\ problem\ size\ and\ work\ done\ would\ be\ 1.$
$The\ total\ number\ of\ levels\ is\ n\ (as\ the\ level\ of\ the\ root\ is\ 0). Therefore,$
$the\ final\ cost\ can\ be\ estimated\ as\ follows$:

$$Total\ Cost = \sum_{i=1}^{n} 1 = 1 + 1 + 1 + \cdots n\ times = 1 \times n = n.$$

$Therefore, the\ asymptotic\ complexity\ would\ be\ O(n).$

$3) Average\ Case\ (Type\ 1)$

$For\ average\ case\ we\ consider\ the\ average\ case\ of\ linear\ search\ , as\ this\ is\ a\ linear\ searching.$

$Here\ say\ we\ have\ no\ such\ estimation\ to\ get\ the\ destined\ element\ from\ the\ list, as\ `Key`\ can\ be\ any\ element.$

$Let\ there\ be\ `k`\ random\ variable\ which\ is\ the\ Key\ \ for\ which\ we\ need\ `k`\ number\ of\ comparisons = \sum_{k=1}^{n} k$

$Hence\ Probability\ `P_k`\ of\ getting\ destined\ element\ is\ equally - likely\ , where\ k = 1, 2, 3, \ldots . k, then:$

$p_1(probability\ of\ getting\ 1st\ element) = \frac{1}{n}$

$p_2(probability\ of\ getting\ 2nd\ element) = \frac{1}{n}$

.

.

$p_k(probability\ of\ getting\ k^{th}\ element) = \frac{1}{n}$

$Hence, P_k = p_1 = p_2 = \cdots = p_k = \frac{1}{n}$

$Where\ `n`\ is\ the\ total\ number\ of\ variables.$

$Then\ , the\ equation\ we\ get:$

$$\Rightarrow \sum_{k=1}^{n} k \times P_k$$

$$We\ know: \sum_{k=1}^{n} k = \frac{n(n+1)}{2} \quad and\ P_k = \frac{1}{n}$$

$$\therefore \sum_{k=1}^{n} k \times P_k \Rightarrow \frac{n(n+1)}{2} \times \frac{1}{n} \Rightarrow \frac{n+1}{2}$$

$$Hence\ general\ result\ \ for\ \ Searching\ `Key`\ in\ Linked\ List\ recursively\ \ is: \frac{n+1}{2}$$

$$\therefore Applying\ Big-\ \Theta\ in\ the\ equation\ we\ get:$$

$$\Rightarrow \Theta\left(\frac{n+1}{2}\right)$$

$$\Rightarrow \Theta\left(\frac{n}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

$$\Rightarrow \frac{1}{2} \times \Theta(n) + \Theta(1)$$

$$\Rightarrow \Theta(n) + \Theta(1)$$

$$\Rightarrow \Theta(n)$$

$$\therefore Average\ Case\ for\ Searching\ `Key`\ in\ Linked\ List\ recursively\ \ is: \Theta(n)$$

$Average\ Case\ (Type\ 2)[Alternative\ of\ Type\ 1]$

$The\ probability\ of\ searching\ element\ from\ Node\ 1\ to\ Node\ N\ is \frac{1}{N}$ $, as\ we\ have\ seen$ $earlier\ and\ each\ time\ it\ runs = T(i-1)+1\ , when\ i\ goes\ from\ 1\ to\ N\ , then$:

$We\ can\ say$:

$$Average\ case\ complexity = \frac{1}{n} \times \left(\sum_{i=1}^{N}(T(i-1)+1)\right)$$

$We\ can\ further\ break\ it\ as$:

$$= \frac{1}{n} \times \big((T(0)+1\ ) + (T(1)+1) + (T(2)+1) + \cdots + (T(N-1)+1)\big)$$

$$= \frac{1}{n} \times \Big(\big(T(0)+T(1)+T(2)+\cdots+T(N-1)\big) + (1+1+1+1+\cdots n\ times)\Big)$$

$As\ , we\ know\ , T(0) = 1\ and\ T(1) = 1\ , and$:

$$\begin{bmatrix} T(0) = 1, \\ T(1) = 1, \\ T(2) = T(1)+1 = 1+1 = 2, \\ T(3) = T(2)+1 = 2+1 = 3, \ldots., \\ T(N-1) = T(N-2)+1 = N-2+1 = N-1, \\ hence\ applying\ it\ we\ get: \end{bmatrix}$$

$$= \frac{1}{n} \times \big((1+1+2+\cdots+N-1)+N\big)$$

$$= \frac{1}{n} \times \left(1+\sum_{i=1}^{N} i\right)$$

$$= \frac{1}{n} \times \left(1 + \frac{N(N+1)}{2}\right)$$

$$= \frac{1}{N} + \frac{N+1}{2}$$

Applying Big Θ ***, we get***:

$$= \Theta\left(\frac{1}{N}\right) + \Theta\left(\frac{N}{2} + \frac{1}{2}\right)$$

$$= \Theta\left(\frac{1}{N}\right) + \frac{1}{2} \times \Theta(N+1)$$

$$= \Theta\left(\frac{1}{N}\right) + \Theta(N) + \Theta(1)$$

$$= \Theta\left(\frac{1}{N}\right) + \Theta(N)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow \Theta(N)$ ***grows linearly with*** `$N$` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ ***decreases to a constant , i.e. when N grows larger ,*** $\frac{1}{N}$ ***approaches to*** 0.

This we can prove by limit and continuity also:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Longrightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0 \text{ , where } c \text{ is constant}\right].$$

As $\Theta(N)$***'s growth rate is higher , therefore , average case is*** $\Theta(N)$.

$$Hence, Average\ Time\ Complexity\ is:\ \Theta(N).$$

$Average\ Case\ (Type\ 3)[Alternative\ of\ Type\ 1\ and\ 2]$

$The\ probability\ of\ searching\ element\ from\ Node\ 1\ to\ Node\ N\ is \frac{1}{N}$ $, as\ we\ have\ seen\ earlier\ and\ each\ time\ it\ runs = T(i-1)+1$ $, when\ i\ goes\ from\ 1\ to\ N$ $, then$:

$We\ can\ say$:

$$Average\ case\ complexity = \frac{1}{n} \times \left(\sum_{i=1}^{N} (T(i-1)+1)\right)$$

$We\ can\ further\ break\ it\ as$:

$$= \frac{1}{n} \times \big((T(0)+1) + (T(1)+1) + (T(2)+1) + \cdots + (T(N-1)+1)\big)$$

Now $, doing\ forward\ substitution$:

$$= \frac{1}{n} \times \big((T(0)+1) + (T(0)+1+1) + (T(0)+1+1+1) + \cdots + (T(0)+1+1+..n\ times)\big)$$

$Note$ $, we\ already\ seen\ this\ in\ substitution\ method$ $, that\ T(N-1)+1 = T(0)+1+1+\cdots n\ times.$

$$= \frac{1}{n} \times \big((T(0)+1) + (T(0)+2) + (T(0)+3) + \cdots + (T(0)+n)\big)$$

$$= \frac{1}{n} \times (n \times T(0) + 1 + 2 + 3 + \cdots + n)$$

$As, we\ know\ , T(0) = 1\ , hence$:

$$= \frac{1}{n} \times (n \times T(0) + 1 + 2 + 3 + \cdots + n)$$

$$= \frac{1}{n} \times \left(n \times 1 + \sum_{i=1}^{N} i \right)$$

$$= \frac{1}{n} \times \left(n + \frac{n(n+1)}{2} \right)$$

$$= 1 + \frac{n+1}{2}$$

$\therefore\ Applying\ Big\ \Theta\ , we\ get$:

$$= \Theta(1) + \Theta\left(\frac{n+1}{2}\right)$$

$$= \Theta\left(\frac{n+1}{2}\right)$$

$$= \Theta\left(\frac{n}{2} + \frac{1}{2}\right)$$

$Taking\ out \frac{1}{2}\ we\ get$:

$$= \frac{1}{2} \times \Theta(n+1)$$

$$= \Theta(n) + \Theta(1)$$

$$= \Theta(n)$$

$\therefore$ ***Average Time Complexity of Searching Key in Linked List is***: $\Theta(n)$

Now among these approaches it is clear there is a constant `C` adding with : $\frac{n+1}{2}$,

though $\frac{1}{N}$ ***is not a constant term but converges to*** 0 ***, will give insignificant values ,***

compared to $\frac{n+1}{2}$ ***, also*** 1 ***is a constant in Type*** 3 ***scenario , basically in worst case***

we found out that $T(N-1)+1 = O(N)$***, similarly for*** i ***from*** 1 ***to*** N ***, we will have***

$O(i)$ ***i.e.*** $\frac{1}{N} \times \sum_{i=1}^{N} (T(i-1)+1) = \frac{1}{N} \times \sum_{i=1}^{N} \big(O(i)\big)$ ***or simply*** , $\frac{1}{N} \times \sum_{i=1}^{N} i$,

the <u>***Type 1***</u> ***approach is more clearer and simpler.***

Note , by relation: $\Omega \leq \Theta \leq O$ ***, we get***:

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$

Singly Linked List	***Cases***	***Time Complexity***
Search Element In SortedList Recursive	$1.$ ***Best Case***	$\Omega(1)$
	$2.$ ***Worst Case***	$O(n)$
	$3.$ ***Average Case***	$\Theta(n)$

****